

聊天模块 (Chat) 详细报告

一、需求与设计

1.1 功能需求

需求编号	需求描述	优先级
CHAT-01	私聊会话创建与消息收发	P0
CHAT-02	群聊创建、成员邀请、消息收发	P0
CHAT-03	消息类型: TEXT / IMAGE / VOICE / SYSTEM / RECALLED / BURN / TIME_CAPSULE / POLL / AI	P0
CHAT-04	消息Reaction (表情回应)	P1
CHAT-05	消息撤回 (2分钟内)	P1
CHAT-06	消息回复 (引用回复)	P1
CHAT-07	@提及 + AI助手触发	P1
CHAT-08	投票消息 (!poll 语法)	P2
CHAT-09	时光胶囊 (定时解锁)	P2
CHAT-10	阅后即焚 (刮开显示)	P2
CHAT-11	语音消息 (MediaRecorder录制)	P1
CHAT-12	草稿自动保存	P2
CHAT-13	多选删除消息	P2
CHAT-14	清除聊天记录	P2
CHAT-15	聊天历史搜索 + 导出HTML	P1
CHAT-16	会话列表 (未读计数、筛选)	P0
CHAT-17	群公告编辑	P1
CHAT-18	群设置 (备注/静音/背景)	P1
CHAT-19	群聊心境天气 (Mood Weather)	P2
CHAT-20	词云生成	P2

1.2 设计决策

- 1. **三层架构**: ChatResource (REST) → ChatService (业务) → ChatDao (SQL)
- 2. **WebSocket + REST 双重通道**: 消息实时推送优先 WebSocket, 失败降级 REST
- 3. **Cursor分页**: history 查询使用 `beforeId` 游标, 避免 OFFSET 大偏移量问题
- 4. **软删除**: 消息不物理删除, 通过 `message_visibility` 表标记 `hidden=1`
- 5. **群组三级权限**: OWNER → ADMIN → MEMBER, 操作前检查角色
- 6. **好友关系门禁**: 私聊前检查双方是否为好友
- 7. **AI助手仅在群聊中被@时触发**: `shouldTriggerAssistant()` 检查

二、后端设计

2.1 核心文件清单

文件	行数	位置
ChatResource.java	240	src/main/java/com/example/chat/chat/ChatResource.java
ChatService.java	572	src/main/java/com/example/chat/chat/ChatService.java
ChatDao.java	569	src/main/java/com/example/chat/chat/ChatDao.java
SendMessageRequest.java	27	src/main/java/com/example/chat/chat/SendMessageRequest.java
GroupCreateRequest.java	6	src/main/java/com/example/chat/chat/GroupCreateRequest.java
GroupInvitePolicy.java	11	src/main/java/com/example/chat/chat/GroupInvitePolicy.java
ChatHistoryPageRequest.java	12	src/main/java/com/example/chat/chat/ChatHistoryPageRequest.java
HtmlHistoryExporter.java	49	src/main/java/com/example/chat/chat/HtmlHistoryExporter.java

2.2 ChatResource — REST API 端点（28个）

会话管理：

HTTP	路径	功能
POST	/api/chat/private	创建/获取私聊会话
GET	/api/chat/conversations	获取用户所有会话列表
GET	/api/chat/conversations/{id}/history	分页获取聊天历史
POST	/api/chat/conversations/{id}/clear	清除聊天记录
GET	/api/chat/conversations/{id}/export	导出聊天历史HTML

群组管理：

HTTP	路径	功能
POST	/api/chat/groups	创建群组
GET	/api/chat/groups/{id}/members	查看群成员
GET	/api/chat/groups/{id}/settings	查看群设置
PUT	/api/chat/groups/{id}/settings	更新群设置
POST	/api/chat/groups/{id}/members	邀请成员
PUT	/api/chat/groups/{id}/name	重命名群
PUT	/api/chat/groups/{id}/members/{mid}/role	设置成员角色
DELETE	/api/chat/groups/{id}/members/{mid}	移除成员
POST	/api/chat/groups/{id}/leave	退出群组
GET	/api/chat/groups/{id}/announcement	获取群公告
PUT	/api/chat/groups/{id}/announcement	更新群公告

消息操作：

HTTP	路径	功能
POST	/api/chat/messages	发送消息
POST	/api/chat/messages/{id}/reactions	添加Reaction
POST	/api/chat/messages/{id}/recall	撤回消息

HTTP	路径	功能
POST	/api/chat/messages/{id}/burn-read	标记阅后即焚已读
GET	/api/chat/messages/{id}/read-by	查看已读成员
DELETE	/api/chat/messages/{id}/reactions/{emoji}	移除Reaction
POST	/api/chat/messages/hide	批量隐藏消息

群邀请 + 特色功能:

HTTP	路径	功能
GET	/api/chat/group-invitations	查看群邀请
POST	/api/chat/group-invitations/{id}/accept	接受邀请
POST	/api/chat/group-invitations/{id}/reject	拒绝邀请
GET	/api/chat/conversations/{id}/wordcloud	词云数据
GET	/api/chat/conversations/{id}/mood-weather	心境天气

代码位置: src/main/java/com/example/chat/chat/ChatResource.java (240行)

2.3 ChatService — 核心业务逻辑

消息发送流程 (send() 方法):

1. 校验 type 不为空 → 默认 "TEXT"
2. ensureCanSend() - 检查会话成员身份
3. 私聊检查双方是否为好友
4. normalizeTimeCapsuleRequest() - 时光胶囊验证(unlockAt范围1分钟~1年)
5. saveMessage() - 写入 messages 表
6. shouldTriggerAssistant() - 群聊中@AI助手 → 调用AI生成回复
7. 返回 [用户消息, (AI回复)]

代码位置: src/main/java/com/example/chat/chat/ChatService.java 第~120行

权限控制体系 (三个层次):

```
// 1. requireGroupMember - 只读操作（查看成员、设置、公告、心情）
// 2. requireManager - 管理操作（邀请、重命名、移除成员、更新公告）
// 3. requireOwner - 最高操作（设置成员角色为ADMIN）
```

代码位置: src/main/java/com/example/chat/chat/ChatService.java 第~450-480行

群邀请策略 (GroupInvitePolicy):

- 如果被邀请者已将邀请者标记为"亲密好友" → ADD_DIRECTLY (直接入群)
- 否则 → CREATE_INVITATION (创建PENDING邀请)
- 代码位置: src/main/java/com/example/chat/chat/GroupInvitePolicy.java (11行)

心境天气算法 (buildMoodWeather()):

- 扫描最近80条消息, 分析关键词模式
- 问号数量 → pressure, 感叹号 → energy, 正面/负面词汇 → positive/negative
- 输出: quiet / storm / sunny / fog / breeze / cloudy
- 代码位置: src/main/java/com/example/chat/chat/ChatService.java 第~500-570行

2.4 ChatDao — 数据访问

核心SQL操作:

方法	关键SQL
<code>createPrivateConversation()</code>	检查已存在的PRIVATE会话(DISTINCT)，无则INSERT
<code>history()</code>	游标分页: <code>WHERE m.id < :beforeId</code> + 子查询排序
<code>conversations()</code>	大复合查询: JOIN + 子查询计算unread_count
<code>saveMessage()</code>	INSERT messages + INSERT IGNORE message_visibility
<code>recallMessage()</code>	UPDATE recalled_at WHERE sent_at >= NOW()-2min

代码位置：`src/main/java/com/example/chat/chat/ChatDao.java` (569行)

三、数据库设计

3.1 conversations 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	会话ID
type	VARCHAR(20)	NOT NULL	PRIVATE / GROUP
title	VARCHAR(80)	—	会话标题
created_at	TIMESTAMP	NOT NULL	创建时间

建表SQL：`src/main/resources/schema.sql` 第71-76行

3.2 messages 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	消息ID
conversation_id	BIGINT	FK → conversations	所属会话
sender_id	BIGINT	FK → users	发送者
type	VARCHAR(20)	NOT NULL	TEXT/IMAGE/VOICE/AI/SYSTEM/RECALLED/BURN/TIME_CAPSULE/POLL
content	TEXT	NOT NULL	消息内容
media_id	BIGINT	—	关联媒体文件
reply_to_message_id	BIGINT	FK → messages(self)	被回复消息
recalled_at	TIMESTAMP	NULL	撤回时间
unlock_at	TIMESTAMP	NULL	时光胶囊解锁时间
sent_at	TIMESTAMP	NOT NULL	发送时间

建表SQL：`src/main/resources/schema.sql` 第90-104行

3.3 message_visibility 表

列名	类型	约束	说明
message_id	BIGINT	PK, FK → messages	消息ID

列名	类型	约束	说明
user_id	BIGINT	PK, FK → users	用户ID
hidden	TINYINT(1)	NOT NULL, DEFAULT 0	隐藏标记

建表SQL: [src/main/resources/schema.sql](#) 第106-113行

3.4 message_reactions 表

列名	类型	约束	说明
message_id	BIGINT	PK, FK → messages	消息ID
user_id	BIGINT	PK, FK → users	用户ID
emoji	VARCHAR(16)	PK	表情字符
created_at	TIMESTAMP	NOT NULL	创建时间

建表SQL: [src/main/resources/schema.sql](#) 第125-133行

3.5 conversation_reads 表

列名	类型	约束	说明
conversation_id	BIGINT	PK, FK → conversations	会话ID
user_id	BIGINT	PK, FK → users	用户ID
last_read_message_id	BIGINT	NOT NULL, DEFAULT 0	最后已读消息ID
read_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	读取时间

建表SQL: [src/main/resources/schema.sql](#) 第115-123行

3.6 chat_groups 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	群ID
conversation_id	BIGINT	FK → conversations	对应会话
owner_id	BIGINT	FK → users	群主
name	VARCHAR(80)	NOT NULL	群名
created_at	TIMESTAMP	NOT NULL	创建时间

建表SQL: [src/main/resources/schema.sql](#) 第135-143行 SchemaMigrator添加列: `announcement TEXT`, `announcement_updated_at TIMESTAMP` 代码位置: [src/main/java/com/example/chat/config/SchemaMigrator.java](#) — `ensureGroupAnnouncementColumn()`

3.7 chat_group_members 表

列名	类型	约束	说明
group_id	BIGINT	PK, FK → chat_groups	群ID
user_id	BIGINT	PK, FK → users	用户ID
role	VARCHAR(20)	NOT NULL, DEFAULT 'MEMBER'	OWNER/ADMIN/MEMBER

建表SQL: [src/main/resources/schema.sql](#) 第145-152行

3.8 chat_group_invitations 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	邀请ID
group_id	BIGINT	FK → chat_groups	群ID
inviter_id	BIGINT	FK → users	邀请者
invitee_id	BIGINT	FK → users	被邀请者
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	PENDING/ACCEPTED/REJECTED
created_at	TIMESTAMP	NOT NULL	创建时间

UNIQUE KEY: `uk_group_invite (group_id, invitee_id)` 建表SQL: `src/main/resources/schema.sql` 第154-165行

四、消息类型设计

Type	content 格式	前端渲染	特殊行为
TEXT	纯文本	气泡卡片	@提及检测 → 脉冲动画
IMAGE	图片URL	图片缩略图	点击放大
VOICE	音频URL	音频播放条	MediaRecorder录制
SYSTEM	系统通知文本	居中灰字	入群/退群/角色变更
RECALLED	原始内容	"[消息已撤回]"	recalled_at非空时客户端判断
BURN	加密内容	Canvas刮开图层	刮开>50% → 标记已读
TIME_CAPSULE	原始内容	倒计时/已解锁	unlock_at到期前隐藏内容
POLL	<code>!poll 标题 选项1 选项2</code>	投票卡片	选项计数+百分比
AI	AI回复文本	特殊颜色气泡	@助手名触发

消息类型渲染代码位置：

- `src/main/webapp/assets/js/chat.js` — `messageBody()` 分发函数 (~692行)
- `renderTimeCapsuleMessage()` (~644行)
- `renderBurnMessage()` (~560行)
- `renderPoll()` (~1684行)
- `wireBurnCanvases()` (~860行)

五、前端设计

5.1 文件结构

- `src/main/webapp/assets/js/chat.js` — 1855行，核心聊天模块
- `src/main/webapp/app.html` — 主应用HTML，包含聊天三栏布局
- `src/main/webapp/assets/css/dashboard.css` — 样式，包含聊天相关所有CSS

5.2 三栏布局





代码位置:

- 三栏布局HTML: `src/main/webapp/app.html` — `#chatView` 区域
- 列宽调整: `src/main/webapp/assets/js/chat.js` — `initChatColumnResizers()` (~128行)
- 列宽持久化: `localStorage` `chatColumnWidths`

5.3 核心前端函数

会话管理:

函数	功能	位置 (chat.js行号)
<code>loadConversations()</code>	加载会话列表	~278
<code>renderConversationList()</code>	渲染会话列表	~242
<code>selectConversation(id)</code>	切换活动会话	~1064

消息处理:

函数	功能	位置
<code>sendMessage(payload)</code>	WebSocket发送 + REST降级	~1113
<code>loadHistory(query)</code>	加载聊天历史(50条/页)	~1005
<code>appendMessage(message)</code>	追加单条消息到列表	~723
<code>messageBody(message)</code>	按type分发渲染	~692

Reaction & Recall:

函数	功能	位置
<code>sendReaction(msgId, emoji, remove)</code>	WebSocket发送Reaction	~809
<code>recallMessage(msgId)</code>	撤回消息(2分钟窗口)	~825
<code>renderReactionBar(message)</code>	渲染Reaction按钮条	~545

创新功能:

函数	功能	位置
<code>renderTimeCapsuleMessage()</code>	时光胶囊渲染	~644
<code>renderBurnMessage()</code>	阅后即焚渲染	~560
<code>wireBurnCanvases()</code>	Canvas刮开逻辑	~860
<code>renderPoll()</code>	投票渲染(!poll解析)	~1684
<code>renderWordCloud()</code>	词云Canvas生成	~1816
<code>renderMoodWeather()</code>	心境天气渲染	~1091

语音消息:

函数	功能	位置
<code>startVoiceRecording()</code>	MediaRecorder开始	~1225
<code>stopVoiceRecording()</code>	停止并上传	~1281

函数	功能	位置
<code>preferredVoiceMimeType()</code>	编解码器检测	~1167

群管理:

函数	功能	位置
<code>openGroupManageDialog()</code>	打开群管理弹窗	~583
<code>renderGroupMembers()</code>	渲染成员列表+角色管理	~511
<code>loadGroupAnnouncement()</code>	加载群公告	~1654

5.4 WebSocket 集成

```
// 连接: ws(s)://host/path/ws/chat
// 事件分发 (chat.js 第~5-45行):
socket.addEventListener("message", e => {
  const data = JSON.parse(e.data);
  if (data.event === "REACTION") → updateReactionUI(data);
  if (data.event === "RECALL") → replaceMessage(data.message);
  if (data.event === "GROUP_INVITATION") → showNotification(data);
  if (data.event === "ERROR") → renderFailedMessage(data);
  else → appendMessage(data); // 普通消息
});
```

代码位置: `src/main/webapp/assets/js/chat.js` — `connectChatSocket()` (~5行) 后端:
`src/main/java/com/example/chat/websocket/ChatEndpoint.java` (96行)

六、聊天历史导出

```
// HtmlHistoryExporter.export(title, messages)
// 生成自包含HTML文档（内联CSS，无外部依赖）
// 所有用户内容HTML转义防XSS
// Content-Disposition: attachment 下载
```

代码位置: `src/main/java/com/example/chat/chat/HtmlHistoryExporter.java` (49行) REST端点: GET
`/api/chat/conversations/{id}/export`